Prerequisites

This worksheet builds on all four of the lab activities. It reuses the LED circuit from Lab 2, the servo from Lab 3 and the hall effect sensor and interrupt from Lab 4, and none of these will be explained in detail again here. Out of your Arduino pack, you will need: 1 Arduino, 1 breadboard, 2 A3144 hall effect sensors, 1 magnet, 4 LEDs, 4 220Ω resistors, 2 10$K\Omega$ resistors, 1 servo, and jumper cables. For the final build step you will also need your mini rig, the motor driver and a 9V battery.

Objectives

Over the next three weeks, we will build a working control system for the mini-rig e-bikes that you have manufactured this semester: an Arduino that reads how fast the rider is pedalling and how fast the wheel is turning, and decides how much help the motor should give. By the end, you will have everything you need to design and test your own control system next semester.

Along the way, you will learn and practise a methodical approach: building a system up one piece at a time, and checking that each piece works before adding the next. This way of thinking is critical when we are working with both hardware and software!

The plan

Getting electronics, code and hardware to all work together at once is never an easy task in engineering![1] Whilst we could build up all the electronics, then write the code and connect the rigs straight away, if we have problems it might be hard to tell where they are coming from.

Instead, we will take a methodical approach. First we will cover how the control system will work and go through a sketch together. Then, we will add each component in turn and test it to make sure that it is working correctly. Once we are happy that the hardware and software are working as expected, we will think about how we can modify the control system to be more effective. Table 1 shows roughly where you should be each week, although this is ultimately up to you!

Week	**Phase**	**Do you need your mini rig?**
7	Phase 0: how the control system works	No
	Phase 1: build steps 1–3	No, test the sensors with a magnet
8	Phase 1: build steps 4–5	Only for step 5
9	Phase 2: developing the control system	Yes

Table 1: The plan for weeks 7 to 9.

[1] Or 'integration hell' as it is sometimes called.

Don't worry if your mini rig isn't finished at the start of week 8. Build steps 1 to 4 only need your Arduino, your breadboard and a magnet: wherever a step asks you to turn the crank or the wheel, you can pass the magnet past the sensor by hand instead. When your rig is ready, 'Moving onto your mini rig' (page 11) explains how to move your sensors onto it.

0 Phase 0: how the control system works

0.1 The big picture

An e-bike motor shouldn't run all the time. In the UK, an electrically assisted pedal cycle may only help the rider while they are pedalling, and must stop helping once the bike reaches 25 km/h. Our control system therefore needs to know two things: how fast the rider is pedalling, and how fast the bike is going. It gets these from two hall effect sensors, one watching a magnet on the crank and one watching a magnet on the wheel (Figure 1), exactly as in Lab 4.

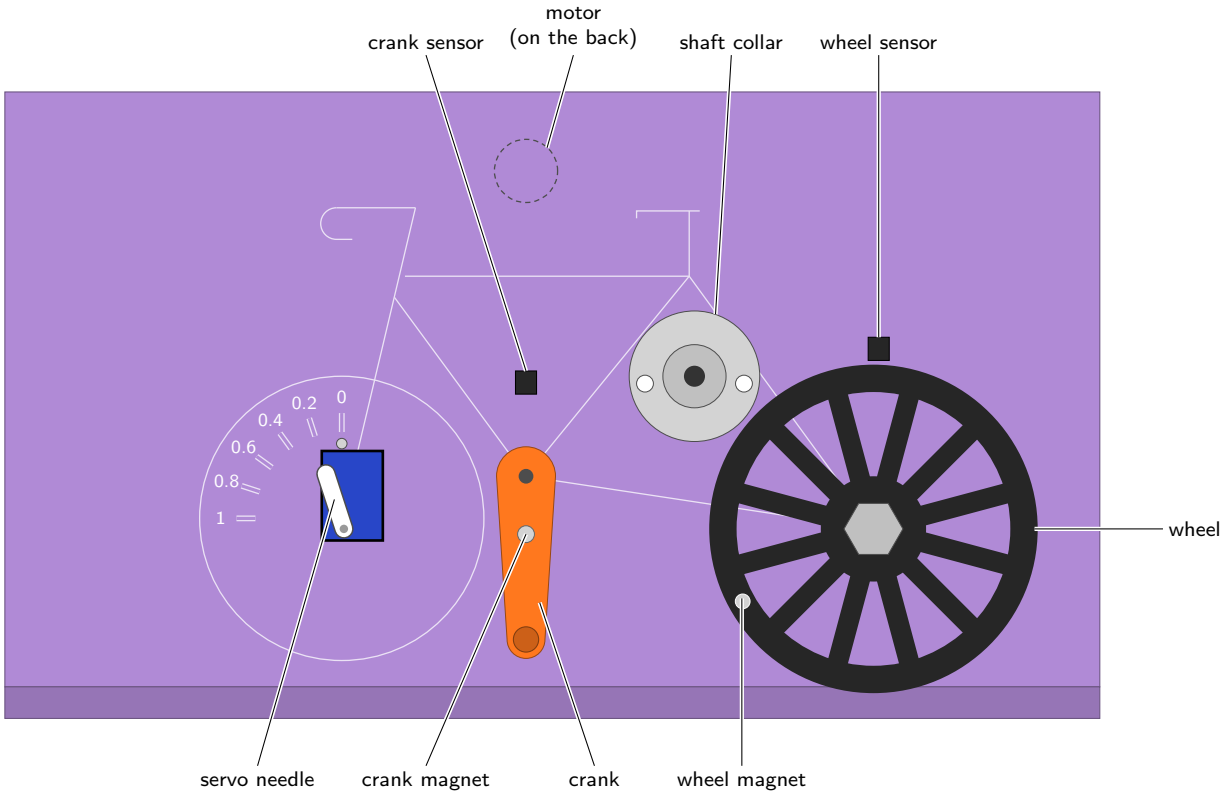


Figure 1: The mini rig, seen from the front. The motor is mounted on the back of the panel.

From these two measurements, the Arduino decides on a *motor demand*: a number between 0 (motor off) and 1 (full power). On the mini rig, this demand drives the motor through a motor driver, and is also shown by a servo acting as a needle (Figure 2).

There are also four LEDs, which we will use to check each part as we build it. The ready LED comes on once the sketch has started up. The crank and wheel LEDs flash each time their sensor

sees the magnet, and the motor LED is on whenever the sketch wants the motor to help. These LEDs will help us ensure each part of our control system is working as intended.

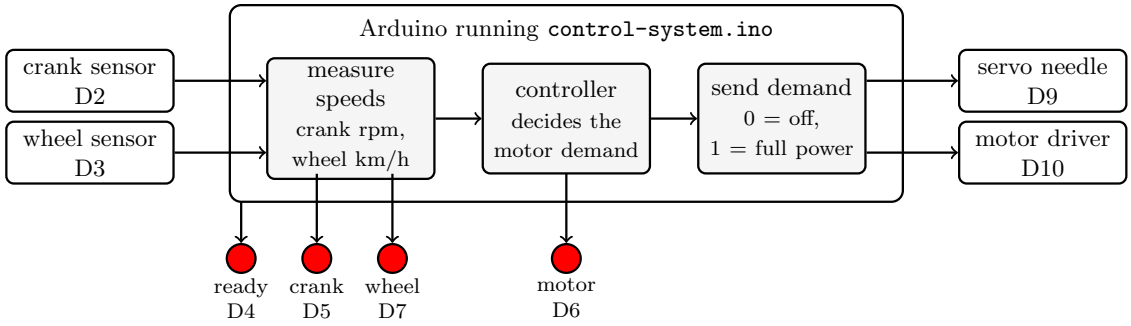


Figure 2: The control system, with the Arduino pin for each part. The LEDs hang off the part of the sketch that switches them.

0.2 The sketch

Unlike the previous labs, this sketch will be provided as-is, and you won't change anything in it until Phase 2. Download `control-system.ino` from `https://github.com/MDCHAMP/MAC233/blob/main/control-system/control-system.ino` and open it in Arduino IDE. This section walks through what each part does and concludes with a check of your understanding.

0.2.1 The parts you must not change

The top of the sketch is fenced off with a `DO NOT CHANGE THESE VALUES` box. The first part names the pins. These match the wiring of the big rigs, so they are fixed:

```
#define SYSTEM_READY_LED_PIN 4                    //
#define CRANK_ACTIVITY_LED_PIN 5                  //
#define MOTOR_ACTIVITY_LED_PIN 6                  //
#define WHEEL_ACTIVITY_LED_PIN 7                  //
#define MOTOR_LOAD_PIN 10                         //
#define CRANK_PIN 2                               //
#define WHEEL_PIN 3                               //
#define ESC_PIN 9                                 //
```

The second part holds the safety limits. The two `ACTIVITY_DELAY` values are the 500-millisecond LED windows from Lab 4. The two `MAXIMUM_DELAY` values say that if a sensor hasn't seen its magnet for 3 seconds, that part of the bike is treated as stopped. `WHEEL_MAXIMUM_SPEED` is the 25 km/h assist limit, and `WHEEL_CIRCUMFERENCE` is the distance a real bike wheel travels in one turn, in metres:

```
#define CRANK_PASS_ACTIVITY_DELAY 500             //
```

```
#define CRANK_PASS_MAXIMUM_DELAY 3000                    //
#define WHEEL_PASS_ACTIVITY_DELAY 500                    //
#define WHEEL_PASS_MINIMUM_DELAY 200                     //
#define WHEEL_PASS_MAXIMUM_DELAY 3000                    //
#define WHEEL_MAXIMUM_SPEED 25.0                         //
// bike                                                  //
#define WHEEL_CIRCUMFERENCE 2.0734                       //
const float pedal_threshold_rpm = 20;                    //
```

0.2.2 Setup function

Most of the 'setup' function is familiar from the labs: it starts the serial connection, sets the four LED pins to outputs and turns them off, and attaches the servo. The interesting part is at the end:

```
// pause system until ready
delay(startup_time);
// only start listening to the sensors now the system is ready, so that
// anything that happens during start up is never recorded
// crank sensor
pinMode(CRANK_PIN, INPUT_PULLUP);
attachInterrupt(digitalPinToInterrupt(CRANK_PIN), crankInterrupt, FALLING);
// wheel sensor
pinMode(WHEEL_PIN, INPUT_PULLUP);
attachInterrupt(digitalPinToInterrupt(WHEEL_PIN), wheelInterrupt, FALLING);
// activate system ready LED
digitalWrite(SYSTEM_READY_LED_PIN, HIGH);
Serial.println("System Ready");
```

Notice that the sensors' interrupts are only attached *after* the 3-second pause. Anything the sensors see while the system is starting up is never recorded, so the motor can't be switched on by something that happened while the system wasn't ready. Only then does the ready LED come on.

Note: the sensor pins use `INPUT_PULLUP` rather than the `INPUT` from Lab 4. This turns on a resistor inside the Arduino that holds the pin `HIGH` when nothing is connected to it. Without it, a pin with no sensor attached yet 'floats' between high and low, and can trigger the interrupt at random.

0.2.3 Interrupts: counting passes of the magnet

Each sensor has its own interrupt function. Every time the magnet passes, it adds one to a counter and remembers the times of this pass and the one before:

```
void crankInterrupt() {
  crank_rotations_counter = crank_rotations_counter + 1;
  crank_interrupt_previous_time = crank_interrupt_current_time;
  crank_interrupt_current_time = millis();
}
```

The wheel's interrupt, `wheelInterrupt`, does the same, with one extra check: a second pass within 200 milliseconds of the last one is ignored, in case the sensor 'bounces' and reports a single pass twice.

0.2.4 Loop function: from passes to speeds

At the start of every loop, the sketch turns the crank and wheel LEDs on if their sensor has seen the magnet in the last 500 milliseconds, just as in Lab 4. It then works out how fast the crank is turning. The time between the last two passes is the time for one turn, so dividing 60 seconds by it gives the speed in revolutions per minute (rpm):

```
float crank_rpm = 0;
float crank_rotation_time = crank_interrupt_current_time -
↪  crank_interrupt_previous_time;
if (crank_interrupt_previous_time > 0 && crank_rotation_time > 1
    && current_loop_time - crank_interrupt_current_time <
    ↪  CRANK_PASS_MAXIMUM_DELAY) {
  crank_rpm = 60.0 / (crank_rotation_time / 1000);
}
```

The `if` statement only calculates a speed if the sensor has been passed at least twice (one pass on its own can't tell us how long a turn took), and if the last pass was less than 3 seconds ago. Otherwise, the speed stays at zero.

The wheel speed is worked out the same way, then converted to km/h: the wheel's rpm, times 60 minutes, times the circumference of the wheel, gives metres per hour, and dividing by 1000 gives km/h.

0.2.5 The example controller

The middle of the 'loop' function, between the two lines of slashes, is where your control system goes. To start with, it holds a simple example:

```
const float motor_step_up = 0.008;
const float motor_step_down = 0.01;

if (
    // the cranks are turning fast enough to count as pedalling
    crank_rpm > pedal_threshold_rpm
    // and the bike has not yet reached the 25 km/h assist limit
    && wheel_kmph <= WHEEL_MAXIMUM_SPEED
) {
  // ramp the motor up
  digitalWrite(MOTOR_ACTIVITY_LED_PIN, HIGH);
  motor_power = motor_power + motor_step_up;
} else {
  // ramp the motor down
  digitalWrite(MOTOR_ACTIVITY_LED_PIN, LOW);
  motor_power = motor_power - motor_step_down;
}
motor_power = constrain(motor_power, 0.0, 1.0); // change the power output to
↪  the motor
motor_demand(motor_power); // send that power to the motor
```

While the rider is pedalling (the crank is turning faster than 20 rpm) and the bike is below 25 km/h, the motor LED comes on and `motor_power` creeps up by 0.008 every loop. Otherwise, the LED goes off and `motor_power` creeps down by 0.01 every loop. `constrain` keeps it between 0 and 1. Because the change is small each time, the motor speeds up and slows down smoothly over a few seconds, rather than jumping straight between off and full power.

0.2.6 Sending the demand to the motor

Finally, `motor_demand` sends the demand out in two forms: as a servo signal on pin D9 (the motor controller on the big rig, and the needle on the mini rig), and as a PWM signal on pin D10, which drives the motor driver on the mini rig:

```
void motor_demand(float demand) {
  // never send more or less than the motor can take
  demand = constrain(demand, 0.0, 1.0);
```

```
  // servo signal: the ESC on the big rig, the speedometer needle on the mini
  ↪  rig
  esc.writeMicroseconds(esc_neutral + demand * (esc_full_power - esc_neutral));
  // pwm signal: the motor driver on the mini rig
  analogWrite(MOTOR_LOAD_PIN, demand * 255);
}
```

Every second, the 'loop' function also prints a status message to the serial monitor, like this one:

```
motor demand: 0.00 crank count: 0 wheel count: 0 crank rpm: 0.00 wheel kmph: 0.00
```

0.3 Check your understanding

Before you build anything, try to answer these with a partner. You will need the answers in Phase 1.

1. The magnet has passed the crank sensor exactly once. What will `crank_rpm` be, and why?
2. The magnet passes the crank sensor once every 1.5 seconds. What is `crank_rpm`? Will the example controller turn the motor on?
3. The rider stops pedalling. Roughly how long is it before the motor LED goes out?
4. `motor_power` is 1 when the rider stops pedalling. The 'loop' function runs about 35 times a second. Roughly how long does the motor take to stop?
5. The magnet passes the wheel sensor once a second. Roughly what speed, in km/h, will the serial monitor show?

1 Phase 1: building and verifying the electronics

We will now build the electronics in five steps, adding one or two components each time. Upload `control-system.ino` to your Arduino once, at the start of step 1, and then leave it alone: the code stays the same while the hardware grows. At every step, follow the same routine:

Build: add only the new components shown in full colour in the figure. Parts from earlier steps are shown faded.
Predict: before you power anything up, write down what you expect the LEDs and the serial monitor to do. Don't read ahead!
Test: try it, and compare what happens with your prediction.
If not. . .: if something doesn't match, stop and fix it before moving on. Because only one thing has changed since the last step worked, the fault is almost certainly in what you just added.

The figures show one way of laying out the breadboard. Use them as a guide: you can put components in any rows you like, as long as the connections are the same. All the figures use the same colours: red wires carry +5V, black wires are GND, and yellow wires carry signals. The servo and the motor driver keep the colours of their own cables. For every LED, the long leg goes towards the resistor and the short leg into the GND rail.

1.1 Step 1: power and the system ready LED

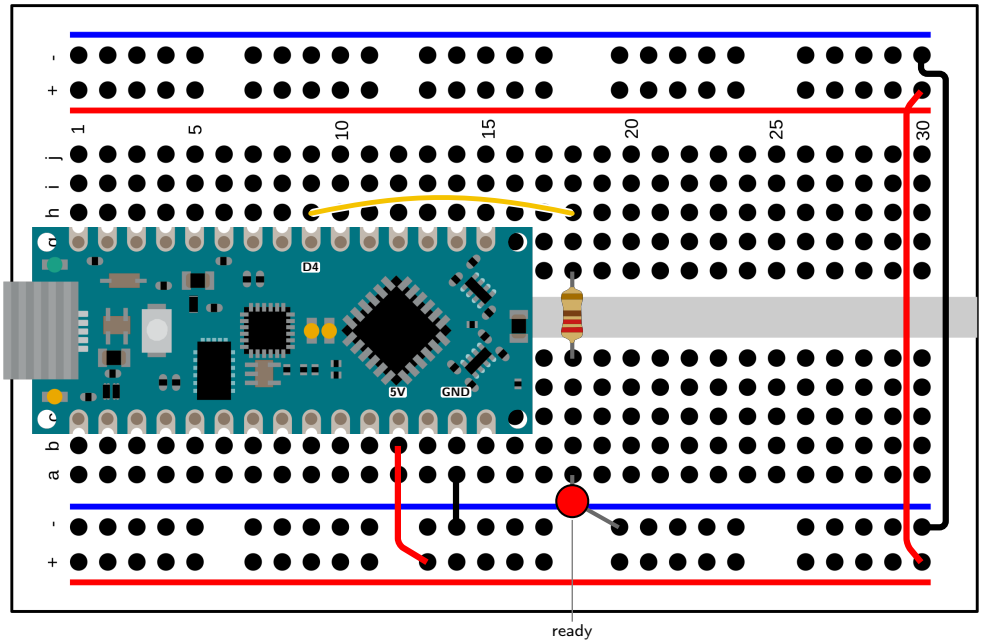


Figure 3: Step 1: power to the rails, and the system ready LED on D4.

Build (Figure 3): place the Arduino across the centre channel of the breadboard. Connect its GND and +5V pins to the − and + rails, and link the rails on both sides of the board so that every rail is powered. Then wire the ready LED as in Lab 2: Arduino D4→220Ω resistor→LED→GND.

Predict: connect the USB cable and upload `control-system.ino`. What should the ready LED do, and when? What should the serial monitor show?

Test: open the serial monitor (9600 baud) and press the reset button on the Arduino. About 3 seconds after reset, the ready LED should come on and stay on, and `System Ready` should appear, followed by a status message every second with every value at zero.

If not...: does the serial monitor show anything at all? If it does, but the LED stays off, is the LED the right way round, and is it in the same row as its resistor?

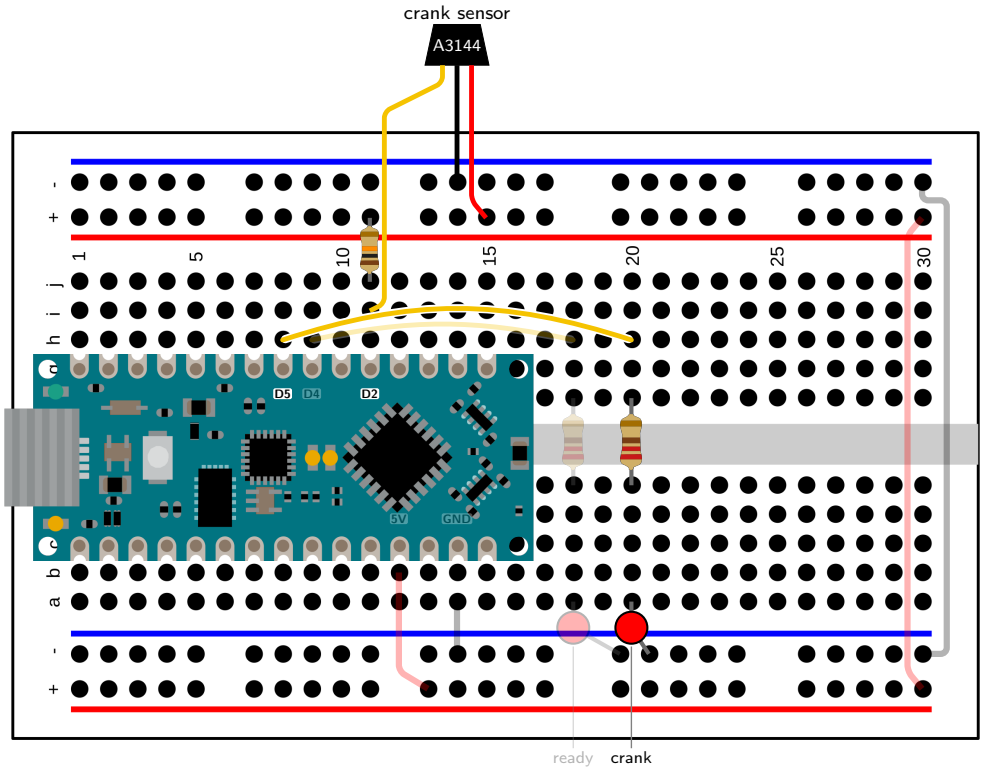


Figure 4: Step 2: the crank sensor on D2, and the crank LED on D5.

1.2 Step 2: the crank sensor and crank LED

Build (Figure 4): wire the crank sensor as in Lab 4: sensor +5V leg→+5V, sensor GND leg→GND, sensor output→Arduino D2, with a $10K\Omega$ resistor between D2 and +5V. The sensors are drawn on leads, as they will be on your rig; until then, connect your sensor's legs to the board with jumper cables. Then wire the crank LED in the same way as the ready LED: Arduino D5→220Ω resistor→LED→GND.

Predict: what will the crank LED do when you pass the magnet past the sensor once? What will `crank count` and `crank rpm` show after one pass? After two passes, a second apart? And what do you think will happen to `motor demand` if you keep passing the magnet about once a second, even though there is no motor connected yet?

Test: pass the magnet slowly past the crank sensor. Every pass should light the crank LED for half a second and add one to `crank count`. `crank rpm` should stay at zero after the first pass, then show about 60 if you pass the magnet again a second later, and fall back to zero 3 seconds after you stop. `wheel count` should stay at zero throughout. If you keep passing the magnet, `motor demand` should climb towards 1, then fall back to zero once you stop. The sketch is already doing its job, even though there is nothing for it to control yet.

If not...: does `crank count` go up, even though the LED stays off? Then the sensor is working,

and the problem is in the LED circuit. If neither changes, try turning the magnet over: the sensor only responds to one pole.

1.3 Step 3: the wheel sensor and wheel LED

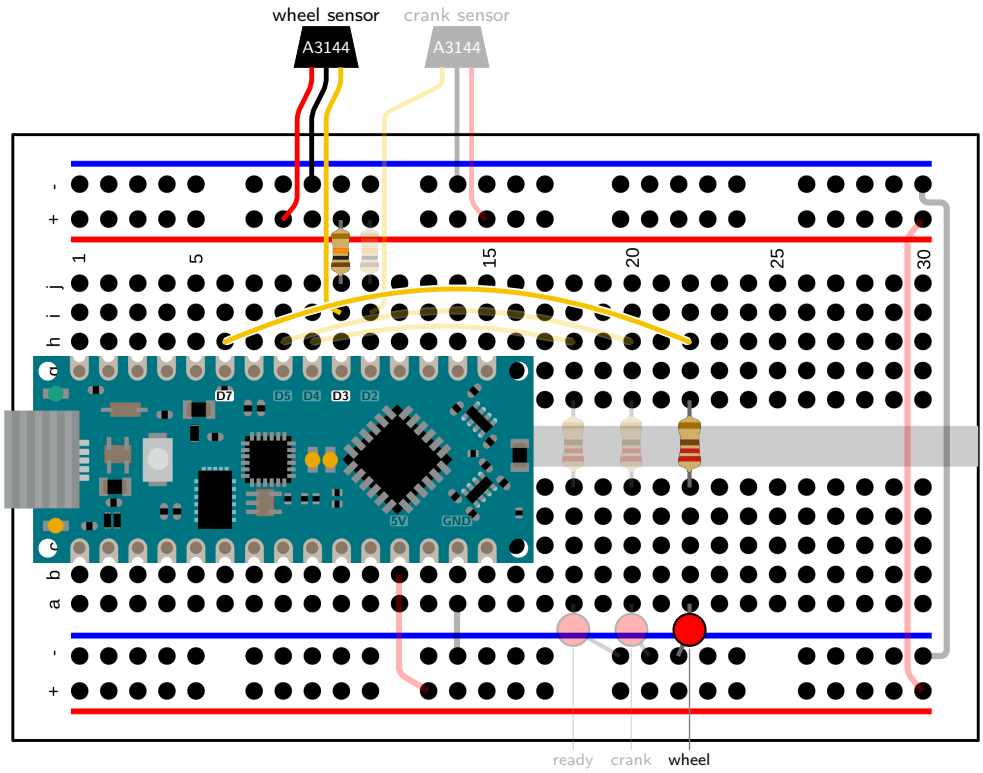


Figure 5: Step 3: the wheel sensor on D3, and the wheel LED on D7.

Build (Figure 5): wire the wheel sensor exactly as you did the crank sensor, but with its output to Arduino D3 and its own $10K\Omega$ resistor between D3 and +5V. Then wire the wheel LED: Arduino D7→220Ω resistor→LED→GND.

Predict: which values on the serial monitor should change when you pass the magnet past the wheel sensor, and which shouldn't? Using your answer to question 5 in Phase 0, what should `wheel kmph` show if you pass the magnet once a second?

Test: each pass should light the wheel LED and add one to `wheel count`, while the crank values stay where they were. One pass a second should give a `wheel kmph` of about 7.5. Then repeat the step 2 test on the crank sensor, to check that it still works.

If not...: does passing the magnet past the wheel sensor change the *crank* values? Check which pin each sensor's output is connected to.

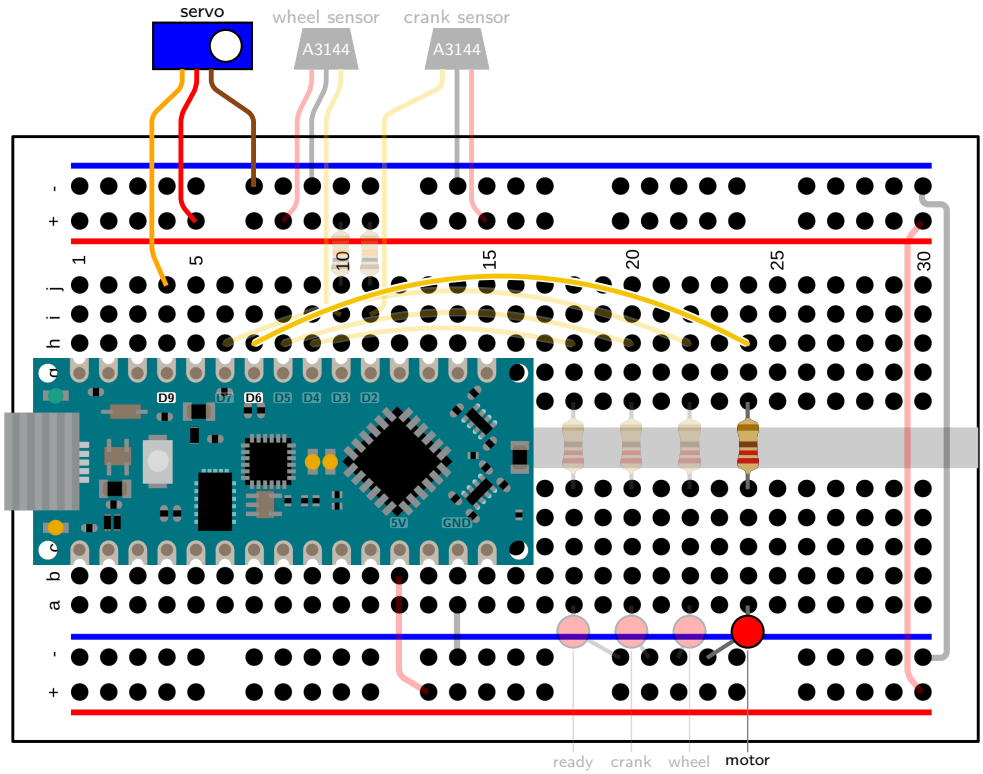


Figure 6: Step 4: the motor LED on D6, and the servo on D9.

1.4 Step 4: the motor LED and the servo

Build (Figure 6): wire the motor LED: Arduino D6→220Ω resistor→LED→GND. Then connect the servo as in Lab 3: signal (orange)→Arduino D9, power (red)→+5V, ground (brown)→GND.

Predict: you pass the magnet past the crank sensor once a second for 10 seconds, then stop. Describe what the motor LED and the servo needle will do, and when. What if you pass the magnet only once every 4 seconds instead?

Test: pedalling, simulated by passing the magnet past the crank sensor about once a second, should turn the motor LED on from the second pass, while the servo needle sweeps up steadily over 3 to 4 seconds. When you stop, the LED should stay on for up to 3 more seconds, and then go off while the needle sweeps back down. Passing the magnet once every 4 seconds should never turn the motor LED on.

If not...: does `motor demand` on the serial monitor rise and fall as expected? If it does, the controller is working and the problem is in the circuit you just added.

Moving onto your mini rig

Once your mini rig is ready, fix the crank and wheel sensors to it so that their magnets pass close by once per turn. The wiring on the breadboard doesn't change. Then repeat the tests from steps

2 to 4, turning the crank and the wheel slowly by hand instead of moving the magnet. Every turn should give exactly one blink of the right LED.

1.5 Step 5: the motor driver (requires mini rig)

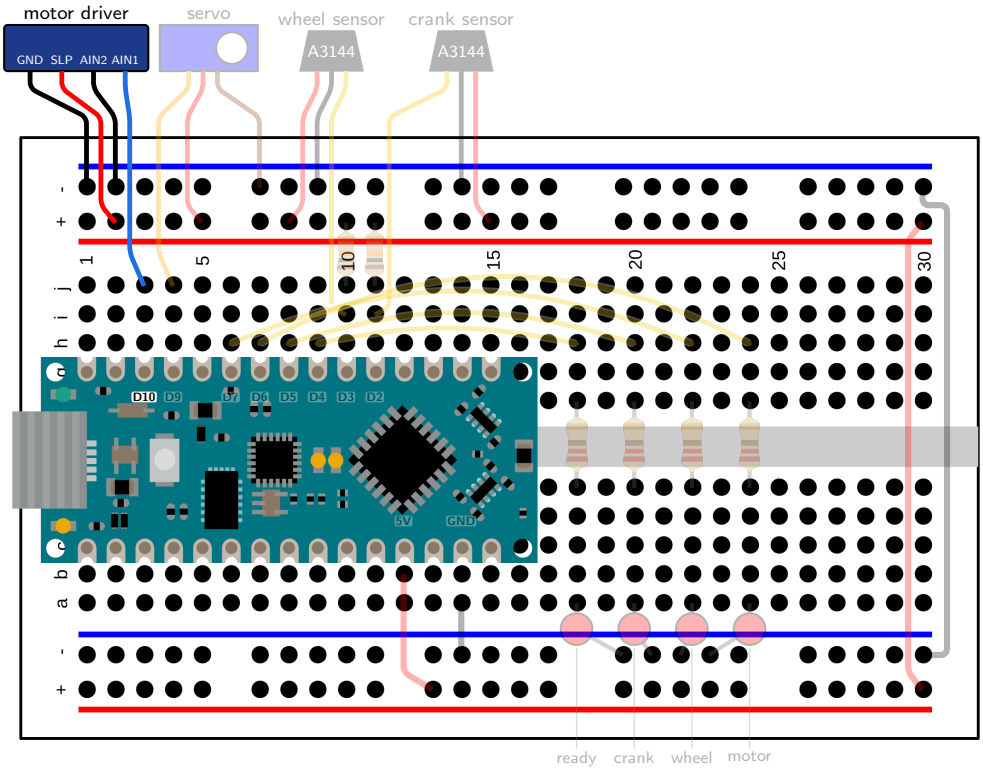


Figure 7: Step 5: the motor driver, controlled by D10.

The Arduino can't power the motor itself: its pins can only supply a tiny current, far less than a motor needs, and a motor can send voltage spikes back that would damage the Arduino. A motor driver sits between the two. It takes its power from a separate supply that is big enough for the motor, and uses the small signal from the Arduino to decide how much of that power to pass on. For this external power supply, we will be using a 9V battery.

Warning: 9V will destroy your Arduino. The 9V battery only ever clips onto the battery snap soldered to the motor driver. Never connect it to the breadboard: the Arduino runs on 5V, and 9V on the +5V rail will damage it permanently. While the battery is connected, don't let the motor driver's pins touch the Arduino or the breadboard either: some of them carry the battery's 9V.

The Arduino tells the driver how hard to run the motor with a PWM signal on D10, set from the motor demand: the higher the demand, the more power the motor gets. Figure 8 shows the driver board and the connections we use.

Build (Figure 7): connect the motor driver: AIN1 (blue)→Arduino D10, SLP (red)→+5V, GND

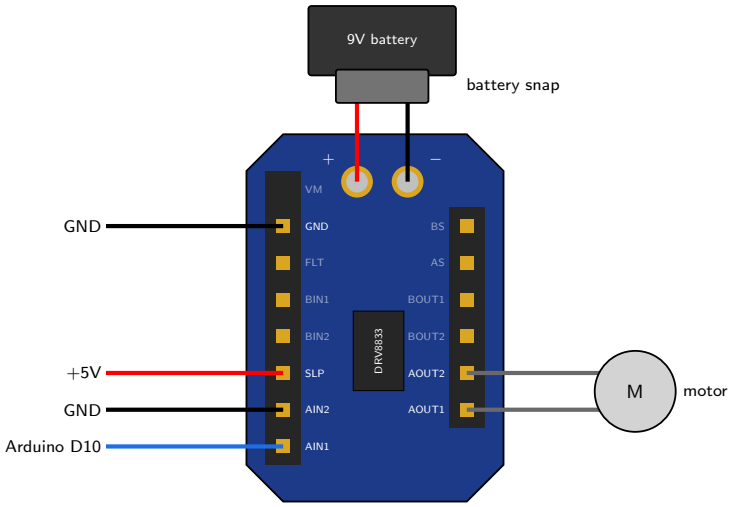


Figure 8: The motor driver board, seen from above. Only the pins with bright labels are used: four come from the breadboard, the 9V battery clips onto the battery snap soldered to the top of the board, and the motor connects to the two outputs on the right.

(black)→GND, AIN2 (black)→GND. Note that the driver has two black wires, and both go to GND. The motor itself and its power supply connect to the driver, not to the breadboard.

Predict: you turn the crank steadily by hand. What will the wheel do, and how will it compare with the servo needle?

Test: turning the crank steadily should make the motor start to drive the wheel, speeding up smoothly as the needle rises. When you stop turning the crank, the motor should slow down and stop as the needle falls.

If not...: does the needle still move as it did in step 4? If it does, the Arduino is sending the right demand, and the problem is between D10 and the motor.

2 Phase 2: developing the control system

With every part built and checked, we can now ask how good the example controller actually is, and how you might improve it.

2.1 Testing the example controller

Turn the crank steadily by hand, at roughly one turn a second, for about 20 seconds, then stop. While you do, watch the servo needle, the wheel and the serial monitor. Afterwards, copy the status messages out of the serial monitor so you can look at how `motor demand` and `wheel kmph` changed over time. Before you start, sketch what you think the two will look like.

2.2 What we want vs what we get

Figure 9 compares what we would like the system to do with what the example controller typically does on a mini rig. Ideally, the motor would give full help for as long as the rider is pedalling, so that the bike speeds up quickly and rides right up to the 25 km/h limit, and it would stop helping as soon as the rider stops. What we get is different in three ways.

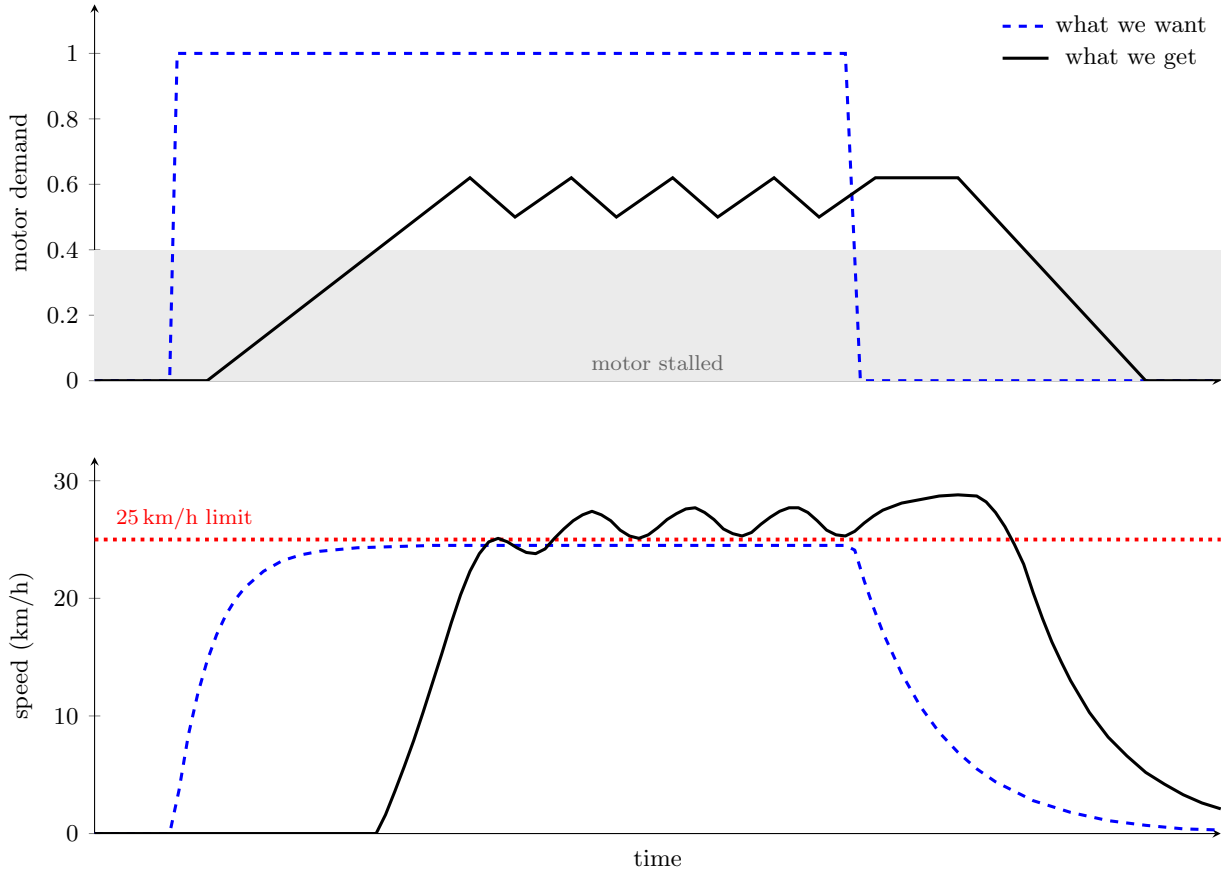


Figure 9: What we want (dashed) and what we typically get (solid) for the motor demand (top) and the bike's speed (bottom), while the rider pedals and then stops. The curves are illustrative: the exact numbers vary a lot from rig to rig.

A slow, delayed start. The demand ramps up from zero, but the motor doesn't turn at all until the demand reaches roughly 0.4. Below that, the motor's power isn't enough to overcome friction in the motor, the gears and the wheel, so the motor is *stalled*. Almost half of the ramp is spent getting nowhere.

Overshoot and wobble around the limit. The controller only knows two things: keep increasing the demand, or decrease it. The bike goes past 25 km/h before the controller notices, because the wheel speed is only measured once per turn, and the wheel's inertia keeps it going for a moment after the demand starts to fall. The speed then drops below the limit, the controller ramps up again, and the cycle repeats.

Help that carries on after the rider stops. The crank speed is only reset to zero 3 seconds after the last pass of the magnet, so the motor keeps helping for up to 3 seconds after the rider stops, and only then ramps down.

2.3 Improving the controller

Now it's your turn. Everything between the two lines of slashes in the 'loop' function is yours to change, but leave the `DO NOT CHANGE THESE VALUES` block alone. Some ideas to start from:

- Start the ramp from just below the point where your motor starts to turn, rather than from zero.
- Instead of only increasing or decreasing the demand, set it from how far the bike is below the speed limit, so that the motor eases off as the bike approaches 25 km/h.
- Change the ramp rates, and see how they affect the overshoot.

Use the same methodical approach as in Phase 1: change one thing at a time, predict what it will do, then test it and compare.

Help

If you have lost track of the sketch or want to check you have the latest version, you can find it at `https://github.com/MDCHAMP/MAC233/blob/main/control-system/control-system.ino`.

The completed breadboard, with every component from steps 1 to 5, is below.

